DA3408 AI Operations: Module 4, Assignment 5

SwiftParcel ETA Serving

Raghava R (DA24B021) November 2026

Machine. Everything was run on an Ubuntu VM (QEMU, aarch64) on a Mac with a 12-core CPU and 18 GB RAM. The VM itself has 6 vCPUs and 5.3 GB RAM, and I used Python 3.14.4. The load generator ran in the same VM as the servers. The commands I used are listed in `README.md`.

1 Part A: REST service

A1. The bundle is loaded once when `eta_app.py` is imported and kept in the module-level variable `model`. Input checks are done with Pydantic: each field has its range, the batch must have 1 to 500 rows, and a model validator rejects a delivery with `distance_km > 150` and `n_stops > 20`. I replaced FastAPI's default 422 response with a handler that returns `validation_failed`, a list of field and message pairs, and the request id. A single middleware handles the request id (echoed if the client sent one, otherwise a new UUID), updates the Prometheus counter and latency histogram (skipping `/metrics`), and turns any unexpected exception into a plain 500. If `model.predict` itself fails, the endpoint catches it and returns the same generic 500.

A2. `test_eta_app.py` has 15 test cases and all of them pass. They cover a valid request, batch order, five invalid inputs (out of range, missing field, wrong type, `rain = 2`, `weight_kg = 0`), the infeasible rule and its boundary, batch sizes 0, 500 and 501, the request id (echoed, generated, and present on a 422), the `/metrics` text, `/healthz`, and a 500 caused by monkeypatching a model that raises.

A3.1 v1 with batch 1, 4 seconds per level:

concurrency	requests	rps	p50 ms	p95 ms	p99 ms	errors	rps × p50 (s)
1	2767	691.8	1.4	1.6	2.3	0	0.97
2	3221	805.2	2.4	3.0	3.8	0	1.93
4	2563	640.8	5.4	10.3	17.3	0	3.46
8	2090	522.5	14.3	25.7	42.0	0	7.47
16	2203	550.8	28.8	46.6	61.5	0	15.86

A3.2 Little's Law says the number of requests in the system is throughput times latency. In the last column rps × p50 comes out close to the concurrency every time, so the load generator really is keeping that many requests in flight. Throughput peaks at concurrency 2 with 805 req/s. After that it stops growing and actually falls to around 520–640 req/s, while p50 goes from 2.4 ms to 28.8 ms, roughly doubling each time the clients double. The extra clients are just waiting in a queue, so I take the saturation point to be concurrency 2 at about 805 req/s. The drop past that point makes sense for this setup: one uvicorn process can only use about one core because of the GIL, and the load generator is fighting it for the same vCPUs. None of the rows break the SLO of p95 $\leq$ 80 ms (the worst is 46.6 ms), so the SLO is not what limits capacity here. The capacity under the SLO is the saturated throughput, about 805 req/s.

A3.3 Batch size at concurrency 1, where ms/row is p50 divided by the batch size:

batch	rps	p50 ms	ms/row
1	694.2	1.4	1.4
100	504.0	1.9	0.019

A request with 100 rows takes only half a millisecond longer than a request with one row, so per row it is about 74 times cheaper (roughly 50,400 rows/s compared to 694). Most of the time for a single row is fixed overhead: the HTTP round trip, parsing JSON, the middleware, building a DataFrame and calling `predict` once. Only validating each row and the vectorised prediction itself grow with the batch. The catch is that every row in the batch has to wait for the whole batch.

A3.4 The change I tried was running two uvicorn workers (`-workers 2`). Each worker is its own process with its own GIL, so two of them can use two cores, and I expected the saturated throughput to roughly double.

	1 worker		2 workers	
concurrency	rps	p95 ms	rps	p95 ms
1	691.8	1.6	622.5	2.1
2	805.2	3.0	752.5	3.7
4	640.8	10.3	1148.8	6.1
8	522.5	25.7	1267.5	11.1
16	550.8	46.6	1233.5	25.6

It did not quite double. The peak went from 805 to 1268 req/s, which is 1.57 times. At concurrency 16 the difference is bigger (2.2 times, and p95 dropped from 46.6 to 25.6 ms) because the single worker was already past its best there. With one or two clients there is no benefit, since there is not enough work in flight to keep both workers busy. I think the ceiling around 1250 req/s comes from the VM: two workers and the load generator share six vCPUs, and the load generator is a single Python process with its own GIL limit.

A3.5 An average barely moves when a few requests are slow (a queue building up, a GC pause), yet those are the requests that users actually notice. A p95 or p99 tells you how slow it gets for the unlucky 5% or 1%, and that is the kind of promise an SLO needs to make.

2 Part B: MLflow

B1. Each bundle is wrapped in a small pyfunc class that returns a DataFrame with `eta_minutes` (rounded to one decimal) and `model_version`. Both are logged with a signature (inputs as float64, so integer JSON values are accepted), a two-row input example, and `pip_requirements` pinned to the versions of mlflow, scikit-learn, pandas, numpy and joblib installed on my machine. The registry is a local SQLite file.

version	model	alias
1	v1: StandardScaler + Ridge	prod
2	v2: GradientBoostingRegressor	candidate

B2. `decide()` does not load anything; it only compares the two metric dictionaries it is given. The MAE gain is measured relative to prod, $(\text{MAE}_{\text{prod}} - \text{MAE}_{\text{cand}})/\text{MAE}_{\text{prod}}$. The assignment does not say whether `max_slice_regression` is relative or absolute, and I treated it as relative: the candidate's MAE on a slice can be at most 5% above prod's. Rush hours are 7–9 and 16–19. `promotion_gate()` loads both aliases, scores them on `make_data(2000, seed=99)`, and times 200 single-row predictions after one warm-up call. `test_gate.py` has 9 tests, mostly placing values just inside and just outside each threshold, and all of them pass.

run	model	MAE	MAE rush	MAE rain	p95 ms	decision
default	prod	11.955	19.68	11.623	0.622	
	cand	6.105	8.58	6.285	0.779	promote
`min_rel_gain=0.60`	prod	11.955	19.68	11.623	0.741	
	cand	6.105	8.58	6.285	0.643	reject (`mae_gain`)

v2 improves the MAE by 48.9%. That clears the default 3% easily, but not the 60% in the second run, so only the gain criterion fails there.

B3. For the shadow test I sent the same 200 single-row requests from `eta_fresh.csv` to both servers. The mean absolute difference between their predictions was 9.48 minutes, 35.0% of requests differed by more than 10 minutes, and p95 latency was 3.5 ms for prod and 3.6 ms for the candidate.

I would promote v2, though through a canary rather than switching everyone at once. On labelled data it nearly halves the error (11.96 to 6.11 minutes), it is better on both slices, and it is as fast as v1. A 35% disagreement rate sounds alarming, but the shadow test has no ground truth, so it only shows that the models disagree. Given the gate results, those large gaps are most likely trips where the linear v1 misses interactions such as distance combined with rush-hour traffic.

Moving the alias does not change a server that is already running. `mlflow models serve` looks up `models:/eta-model@prod` once when it starts, loads that version into memory and never asks the registry again. There are two ways to switch without dropping requests. One is to start a second server on the new version, wait until `/ping` answers, point the load balancer at it, and then stop the old one (a blue-green or rolling switch). The other is to reload inside the service: a background thread checks the alias, loads the new model next to the old one, and swaps the reference once it is ready. Ray Serve works in a similar way when redeploying, since it brings up new replicas before removing the old ones.

3 Part C: Ray Serve sticky canary

C1. `EtaModel` loads its bundle in `__init__`, so each replica loads the model once. Prod has 2 fixed replicas, and the candidate autoscales between 1 and 3 replicas with `target_ongoing_requests=2`, a 5 s upscale delay and a 60 s downscale delay. The FastAPI router sends a request to v2 if it carries `X-Beta: 1` or if `bucket(customer_id) < weight`. `bucket` takes the SHA-256 digest of the customer id and maps its first 8 bytes to a number in $[0, 1)$. Since a customer's number never changes, raising the weight from 0.2 to 0.5 only adds customers to v2 and never moves anyone back. Model replicas ask for 0.25 CPU each and the router for 0.1, so even with 3 candidate replicas the total is $0.1 + 2 \times 0.25 + 3 \times 0.25 = 1.35$ CPUs.

C2. Output of `verify_ray.py`, with Ray limited to 2 CPUs:

```
Ray CPUs: 2.0
(a) weight 0.2: 21.3% of 1000 customers on v2
(b) second call: 200/200 same arm
(c) after redeploy: 200/200 same arm
(d) weight 0.5: 50.9% on v2, moved from v2 back to v1: 0
(e) X-Beta: 1 on 50 prod customers: 50/50 on v2
replicas: {'EtaModelV1': {'RUNNING': 2}, 'EtaModelV2': {'RUNNING': 1}, 'Router': {'RUNNING
    ': 1}}
all checks passed
```

Every check is an assertion in the script, so a failure would stop it before the last line. 21.3% is within the allowed 16–24%.

C3.

1. Python's `hash()` on strings is salted with a random seed each time a process starts. After a redeploy, or on a second router replica, the same customer would get a different value and could switch models. SHA-256 gives the same answer in every process and on every machine.
2. From the customer's side, the ETA for the same parcel could jump between two different values on every refresh, which would make the app look broken. From the analysis side, the experiment compares customers on v1 with customers on v2, and someone who saw both models cannot be counted in either group.
3. A single prediction takes a few milliseconds, so allowing about 2 requests in flight per replica keeps the queue short without adding replicas for every small spike. Scaling up after 5 s reacts quickly, while waiting 60 s before scaling down avoids replicas being added and removed over and over. Three replicas is the most that fits in 2 CPUs next to prod and the router, and replicas beyond the real cores would not add any compute. Keeping at least one replica means canary customers never hit a cold start.

4 Part D: Release gate

D1.1 `smoke_eta.py` runs seven checks and exits with 1 if any of them fails or throws an error:

1. `/healthz` returns status ok;
2. a valid request returns the expected `model_version` and an ETA that is a finite number between 0 and 1440 minutes;
3. a batch of three rows gives the same predictions, in the same order, as sending the rows one by one;
4. sanity check: a 50 km delivery is not predicted to be faster than a 5 km one;
5. sanity check: traffic index 0.9 is not predicted to be faster than 0.1;
6. a request with `distance_km = -5` gets a 422;
7. p95 latency over 50 single-row requests is under `-max-p95-ms` (80 ms by default).

Against the Part A server running v1, all seven passed with `-expect-version v1` and the exit code was 0. Running it with `-expect-version v2` made the version check fail and the exit code was 1, which is what a release gate should do.

D1.2 Abort criteria for a 10% canary of v2, always comparing v2 with v1 over the same period:

metric	abort if	window
5xx error rate of v2	above 1%, or more than twice v1's	5 min, at least 200 requests
p95 latency of v2	above 80 ms, or 1.5 times v1's	two 10 min windows in a row
v2 ETAs outside 1–600 min	more than 0.5% of predictions	15 min
MAE of v2 against actual delivery times, overall and on rush and rain	more than 5% above v1 on any of them	2 h, at least 300 completed deliveries

Not tested

- The smoke test was only run against the Part A server. It also does not work against the MLflow servers, which use a different request format and have no input validation.
- I did not use Docker or `mlflow models build-docker`.
- All the timing numbers come from one VM with the client on the same machine, so they say little about real production capacity.

References and AI use

The code follows the Lecture 10 notebooks in the course repository (`https://github.com/sudarsun/DA3408`): *REST_API_Serving_Exercise* for the service layout (`load_test.py` was used as given), *MLflow_Serving_Exercise* for the pyfunc wrapper, aliases, promotion gate and shadow test, *Ray_Serve_Exercise* for the deployments, router, autoscaling and `extra_work_ms`, and `Monitoring/prom_client.py` for `prometheus_client`.

I used Claude (Anthropic) while writing the code and drafting the answers. I ran every experiment myself, checked the files, and all the numbers in this report are from my own runs.